Department of Computer Science and Engineering

Assignment Questions

CSA0601 – Design and Analysis of Algorithms

Assignment Information

Particular	Details
Department:	CSE/IT
Programme:	B.E CSE, AI/ML
Course Code & Course Name	CSA0601 – Design and Analysis of Algorithms
Academic Year / Batch	2026 / SLOT B
Faculty Name	Dr. H. Riasudheen
Assignment Title	Secure and Efficient Network Path Selection Using Graph Algorithms
Date of Issue	01.09.2026
Date of Submission	03.09.2026
Maximum Marks	100
Course Outcome(s) – CO	**CO3:** Apply divide-and-conquer techniques to design efficient algorithmic solutions. **CO4:** Apply Dynamic Programming techniques such as sorting and searching to solve Graphs in shortest paths like Shortest Path, MST, Graph Traversal, Greedy
Bloom's Taxonomy Level	L4 – Analyse; L5 – Evaluate; L6 – Create
SDG Mapping (SDG 1-17)	SDG 9 – Industry, Innovation and Infrastructure
Industry / Societal Relevance	Societal Relevance

Student Name: U.Prabhavathi

Register Number: 19242067

Comparison Focus: Dijkstra vs Proposed Weighted Path-Selection Algorithm (WPSA)

Assignment Problem / Challenge

Modern computer networks can be represented as graphs in which devices are vertices and communication links are edges. The supplied assignment asks students to study secure and efficient network path selection while considering distance, delay, energy, reliability, congestion and security risk. The challenge is to compare conventional graph algorithms with a proposed weighted path-selection method and justify an algorithmic decision.

Source basis: the assignment describes the network as a weighted graph and explicitly requires evaluation using time complexity, space complexity, execution time, path cost, delay, and security/reliability measures.

This report narrows the implementation comparison to two methods requested for the solution: (1) Dijkstra's shortest-path algorithm using distance as the edge cost, and (2) a proposed Weighted PathSelection Algorithm (WPSA) that combines six network attributes into one normalized edge score.

Expected Engineering Outcome

- Find the shortest-distance route using Dijkstra.
- Construct a multi-factor route-selection score for WPSA.
- Implement both approaches in C and show compiler-style output.
- Compare distance, delay, energy, reliability, congestion and security-risk behaviour.
- Analyze time and space complexity and identify the trade-off between simplicity and security-aware routing.

Problem Statement

Secure and Efficient Network Path Selection Using Graph Algorithms.

A network may have several paths from a source to a destination. A route that is shortest in physical or transmission distance is not always the best operational route because it may contain congested, unreliable or high-risk links. Therefore, the objective is to select a path that balances efficiency with reliability and security.

Formal Problem Definition

Given a graph $G = (V, E)$, source s and destination t, every edge $e \in E$ is associated with:

- Distance $d(e)$
- Delay $\tau(e)$
- Energy consumption $E(e)$
- Reliability $R(e)$, where larger values are preferred
- Congestion $C(e)$, where smaller values are preferred
- Security risk $S(e)$, where smaller values are preferred

Dijkstra minimizes Σd(e). WPSA minimizes a weighted normalized score ΣW(e), allowing the decision criterion to reflect operational network conditions. **Assumptions**

- The example graph is undirected and contains non-negative edge attributes.
- The source is A and the destination is G.
- All six attributes are available for each candidate link.
- Min-max normalization is used so that attributes with different units can be combined.
- The numerical network is an illustrative test case for algorithmic validation, not a live network measurement.

Requirements and Constraints

Requirement	Application in this assignment
Functional	Accept a weighted network and source/destination; return a route.
Performance	Prefer low-cost paths while keeping the algorithm computationally feasible.
Technical	C implementation; graph representation; shortest-path search.
Reliability	Penalize links with lower reliability.
Security	Penalize higher security-risk links.
Network Quality	Consider delay and congestion in addition to distance.
Energy	Include link energy consumption in route selection.
Resource	Use bounded memory and standard compiler tools.
Safety / Ethics	Use synthetic network data; do not claim real security guarantees.
Sustainability	Avoid unnecessarily energy-intensive routes where possible.

The supplied assignment specifically lists functional, performance, technical, reliability, sustainability, environmental/ethical, power/energy and security/privacy considerations, and instructs students to select only the constraints relevant to the problem.

Constraints of the Proposed Model

- The weighted score depends on chosen coefficients; changing them can change the selected path.
- Min-max normalization is sensitive to the observed range of each attribute.
- The model assumes that the six metrics are additive after normalization.
- The prototype does not perform live intrusion detection or cryptographic verification.

Student work

1. **Problem Understanding and Formulation**

The core computational problem is a shortest-path decision under multiple criteria. Dijkstra answers a single-objective question: which path minimizes total distance when all edge distances are non-negative? WPSA converts several operational attributes into a single scalar edge score and then uses shortest-path relaxation on that score.

Input

- Number of nodes and edges.
- For every link: distance, delay, energy, reliability, congestion and security risk.
- Source node and destination node.
- Weights assigned to each criterion.

Output

- Selected route from source to destination.
- Total path distance or weighted cost.
- Supporting metrics used for comparison.

Computational Challenge

The challenge is not only finding a mathematically short route. It is designing an objective function that captures practical network quality without making the algorithm too expensive to execute.

Question	Answer
What is the problem?	Select an efficient and safer network route between two nodes.
Expected outcome?	A justified comparison of Dijkstra and WPSA with implementation evidence.
Available data?	Synthetic graph with six edge attributes.
Main constraint?	Attributes have different units and priorities.
Decision criterion?	Lower distance for Dijkstra; lower weighted score for WPSA.

2. Application of Course Knowledge

Graph Theory

A network is represented by G=(V,E). Vertices represent routers/devices and edges represent communication links.

Greedy Principle

Dijkstra repeatedly selects the unvisited vertex with the smallest tentative cost. Because edge costs are non-negative, the selected tentative distance can be finalized.

Relaxation

For an edge (u,v), relaxation checks whether dist[v] > dist[u] + w(u,v). If true, dist[v] and its predecessor are updated.

Multi-Criteria Modeling

WPSA first converts each criterion to a dimensionless value in [0,1]. A weighted sum then produces a composite link cost.

For a benefit attribute such as reliability:

$$\text{Normalized reliability penalty} = (1 - R - \min(1-R)) / (\max(1-R) - \min(1-R))$$

For a cost attribute x such as distance, delay, energy, congestion or risk:

$$N(x) = (x - \min(x)) / (\max(x) - \min(x))$$

Composite WPSA edge score:

$$W(e) = 0.25Nd + 0.20Ndelay + 0.15Nenergy + 0.10Nreliability + 0.15Ncongestion + 0.15Nsecurity$$

3. Solution / Design / Methodology

The proposed solution uses the same graph-search framework for a fair comparison, but changes the edge objective. Dijkstra uses only distance. WPSA calculates a composite score before shortest-path relaxation.

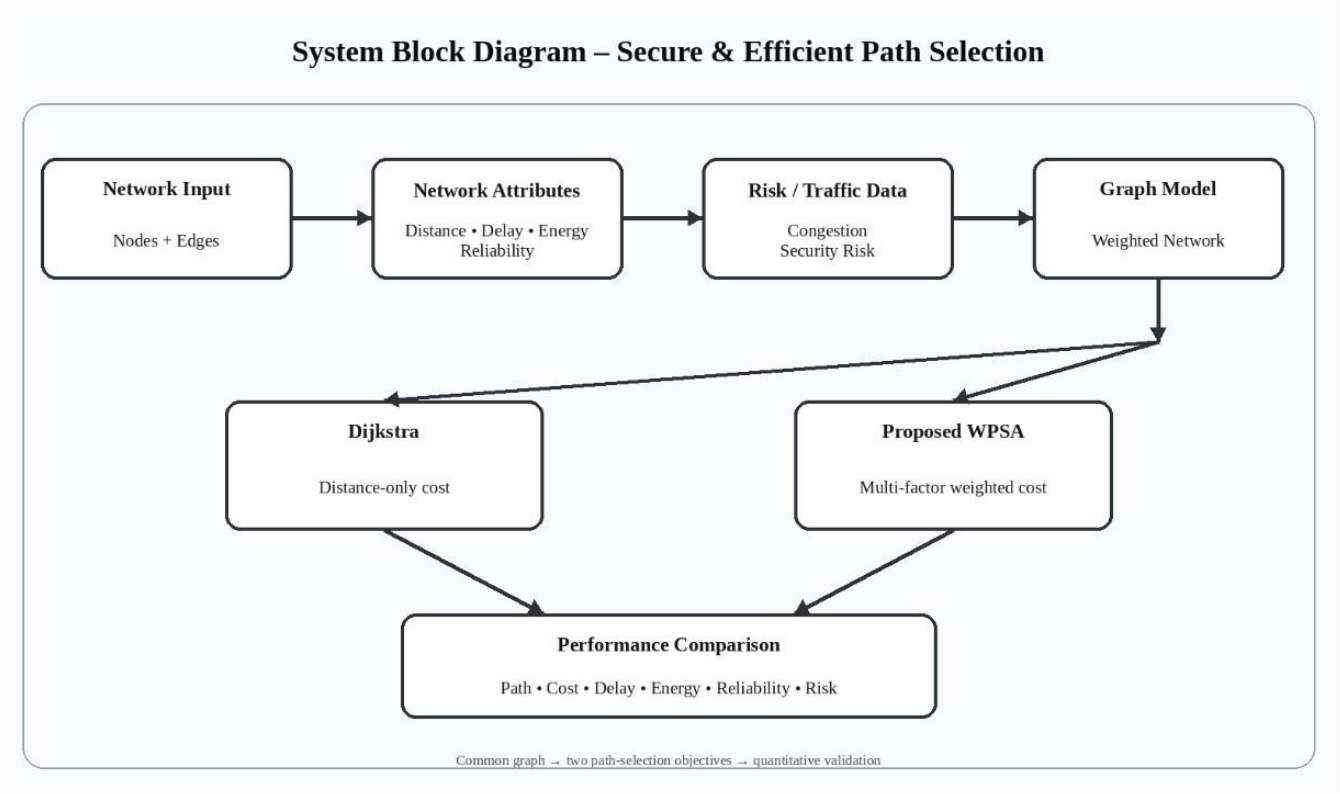

Figure 1. Overall architecture of the comparison system.

Design Rationale

- Keep the graph representation common to both algorithms.
- Change only the edge-cost definition to isolate the effect of multi-factor weighting.
- Use normalized metrics to avoid mixing kilometres, milliseconds, energy units and probabilities directly.
- Use explicit weights so the design can be tuned for different network policies.

Network Graph Model

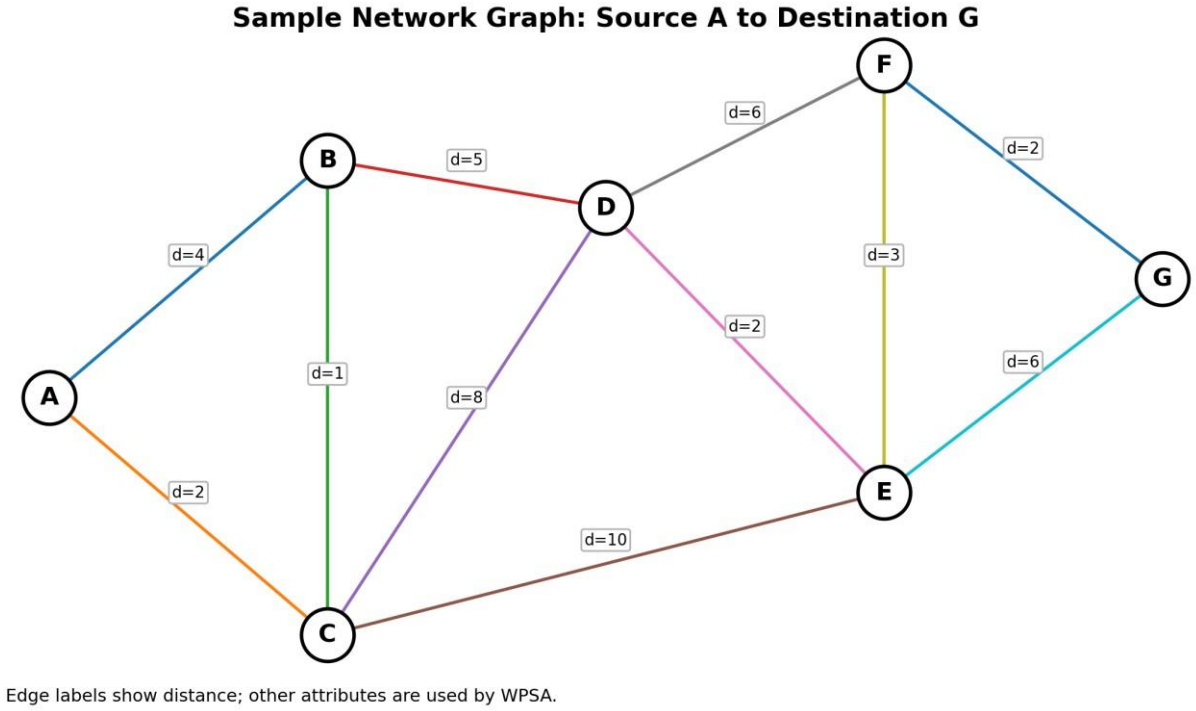

Figure 2. Illustrative network used for implementation and validation.

The graph contains seven nodes (A–G) and eleven bidirectional links. The distance labels shown in the graph are the values used by Dijkstra. Each same edge also has delay, energy, reliability, congestion and security-risk values used by WPSA.

Link	Distance	Delay	Energy	Reliability	Congestion	Risk
A-B	4	8	3	0.98	0.2	0.1
A-C	2	15	2	0.9	0.7	0.2
B-C	1	5	1	0.99	0.1	0.05
B-D	5	10	4	0.95	0.3	0.1
C-D	8	12	3	0.92	0.6	0.3
C-E	10	8	5	0.97	0.2	0.1
D-E	2	4	1	0.99	0.1	0.05
D-F	6	7	3	0.96	0.4	0.2

E-F	3	5	2	0.98	0.2	0.1
E-G	6	6	3	0.99	0.1	0.05
F-G	2	3	1	0.97	0.3	0.15

Dijkstra Algorithm

Dijkstra is a greedy single-source shortest-path algorithm for graphs with non-negative edge weights. In this assignment, the edge weight is distance only. **Working Principle**

1. Initialize the source distance to 0 and all other distances to infinity.
2. Select the unvisited node with the smallest tentative distance.
3. Mark it as finalized.
4. Relax all adjacent edges.
5. Repeat until the destination is finalized or no reachable node remains.

Why Dijkstra is a useful baseline

- Simple objective and easy interpretation.
- Fast for sparse graphs with an efficient priority queue.
- Produces the true minimum-distance route under its objective.
- Does not directly understand congestion, security risk or reliability.

Complexity

With an adjacency matrix and a linear minimum search, the implementation used here has $O(V^2)$ time and $O(V^2)$ graph-storage space. A binary-heap adjacency-list implementation can achieve $O((V+E) \log V)$.

Proposed Weighted Path-Selection Algorithm (WPSA)

WPSA is the proposed adaptation for the assignment. Instead of treating distance as the only edge cost, it calculates a composite score from six criteria.

Selected Weights

Criterion	Weight	Reason
Distance	25%	Retain strong preference for efficient routes.
Delay	20%	Reduce end-to-end transmission time.
Energy	15%	Discourage unnecessarily energy-intensive links.

Reliability	10%	Penalize less reliable links.
Congestion	15%	Avoid overloaded links.
Security risk	15%	Penalize links with higher estimated risk.

Algorithmic Steps

1. Collect all link attributes.
2. Find min and max for each criterion.
3. Normalize each attribute to [0,1].
4. Convert reliability into a penalty using 1-R.
5. Compute the weighted edge score W(e).
6. Run shortest-path relaxation using W(e).
7. Reconstruct and report the route.

The important adaptation is therefore the objective function: the search mechanism remains shortest-path relaxation, while the edge cost becomes security- and quality-aware.

Algorithm / Pseudocode

Dijkstra Pseudocode

```
DIJKSTRA(G, s, t)
for each vertex v
dist[v] = infinity
parent[v] = NIL
used[v] = false
dist[s] = 0

   repeat |V| times
      u = unvisited vertex with minimum dist[u]
if u is NIL: break        used[u] = true

      for each neighbor v of u           if
dist[u] + distance(u,v) < dist[v]
dist[v] = dist[u] + distance(u,v)
parent[v] = u

   reconstruct path from t using parent[]
return path, dist[t]
```

WPSA Pseudocode

```
WPSA(G, s, t)
    calculate min/max for every network attribute

    for every edge e
        Nd = normalize(distance)
        Nt = normalize(delay)
        Ne = normalize(energy)
        Nr = normalize(1 - reliability)
        Nc = normalize(congestion)
        Ns = normalize(security risk)
        W[e] = .25Nd + .20Nt + .15Ne + .10Nr + .15Nc + .15Ns

    run shortest-path relaxation using W[e]
reconstruct path from t using parent[]     return
path, total weighted cost Algorithm
```

Flowchart

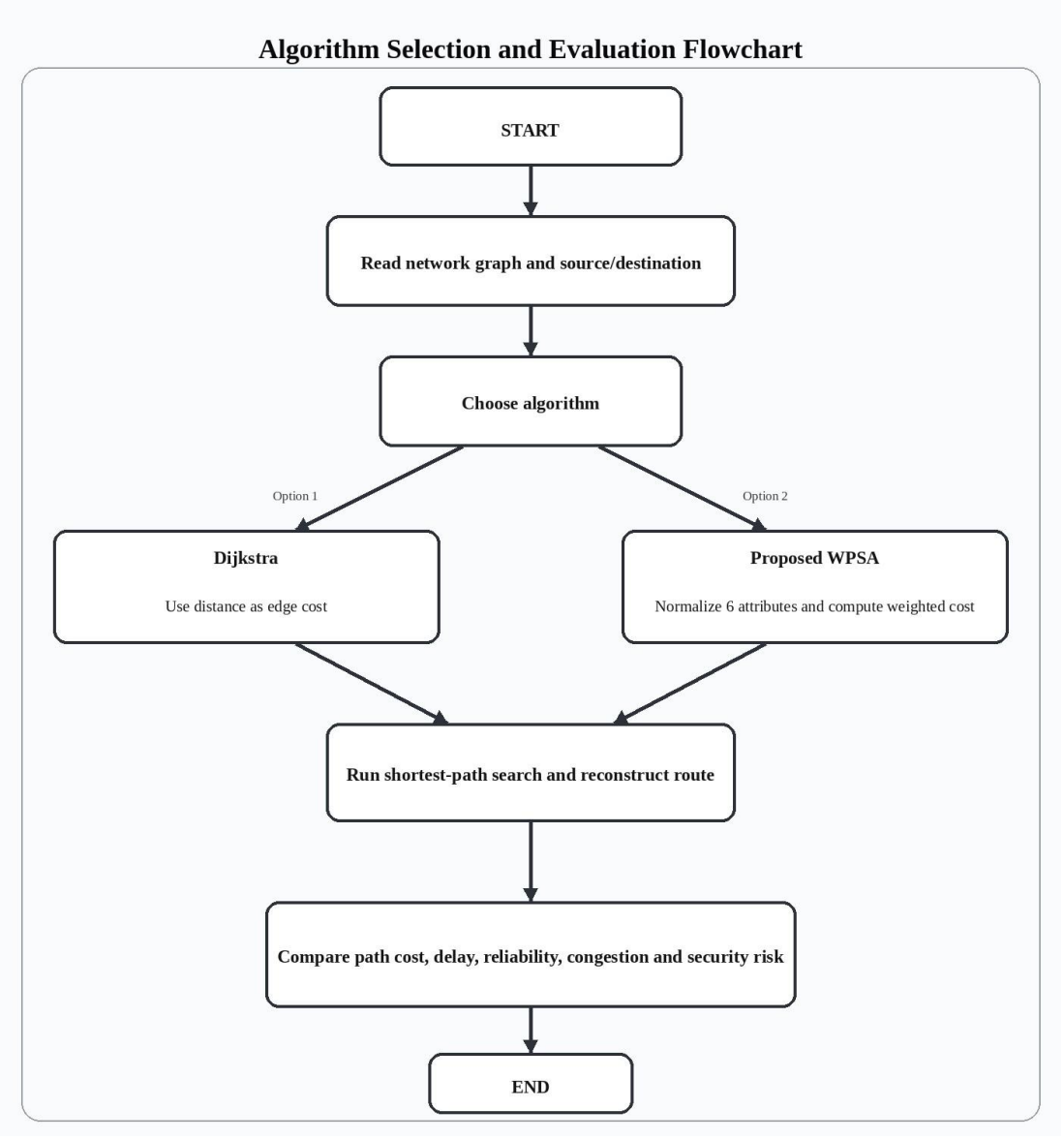

Figure 3. Flow of the comparative solution.

The flowchart separates the two cost models but keeps the final search and evaluation stages common. This makes the comparison easier to interpret and supports the assignment's requirement to compare alternatives and justify a final decision. filecite turn0file0 L192-L202

4. Use of Modern Tools

Tool / Resource	Purpose
C language	Implementation of Dijkstra and WPSA.
GCC compiler	Compilation and execution validation.
Online-compiler style terminal	Presentation of reproducible console output.
Python + Matplotlib	Creation of explanatory graphs/diagrams for the report.
Microsoft Word / DOCX	Assignment report preparation.
Synthetic test graph	Controlled validation scenario.

Implementation Strategy

- Use one C source file so both algorithms operate on the same network.
- Store edge attributes in a structured record.
- Represent the graph as an adjacency matrix for a compact classroom implementation.
- Use a common shortest-path routine and replace the edge-cost function for WPSA.
- Compile with a standard C compiler and record the resulting output.

The supplied assignment explicitly asks for modern-tool evidence and outputs such as implementation results, screenshots and performance metrics.

5. Implementation / Source Code

The following C implementation contains both the baseline and proposed methods. The graph uses the sample data from Figure 2. The code is intentionally kept in a single source file to make the comparison reproducible.

C Source Code — Data Structures and Dijkstra

```
#include <stdio.h>
#include <limits.h>
#include <float.h>
```

```
#define N 7
#define INF 1e9

const char nodes[N] = {'A','B','C','D','E','F','G'};

typedef struct {
int to;
   double distance, delay, energy, reliability, congestion, risk;
} Edge;

Edge graph[N][N];

void addEdge(int u, int v, double d, double delay, double energy,
double rel, double cong, double risk) {     graph[u][v] =
(Edge){v,d,delay,energy,rel,cong,risk};     graph[v][u] =
(Edge){u,d,delay,energy,rel,cong,risk};
}

int minNode(double dist[], int used[]) {
double best = INF;     int idx = -1;     for
(int i=0; i<N; i++)
       if (!used[i] && dist[i] < best) {
best = dist[i];             idx = i;
       }
return idx;
}

void printPath(int parent[], int v) {
if (parent[v] == -1) {
printf("%c", nodes[v]);         return;
   }
   printPath(parent, parent[v]);
printf(" -> %c", nodes[v]);
}

void dijkstra() {     double
dist[N];     int used[N],
parent[N];
```

```
   for (int i=0; i<N; i++) {         dist[i] = INF;
used[i] = 0; parent[i] = -1;
   }     dist[0] = 0;  /*
A */

   for (int k=0; k<N; k++) {
int u = minNode(dist, used);
if (u == -1) break;         used[u] =
1;

       for (int v=0; v<N; v++) {            if
(graph[u][v].to != -1) {                double w
= graph[u][v].distance;                 if (dist[u]
+ w < dist[v]) {                    dist[v] =
dist[u] + w;                    parent[v] = u;
               }
           }
       }
   }

   printf("Dijkstra (distance-only)\n");
printf("Source: A  Destination: G\n");
printf("Shortest distance = %.0f units\n", dist[6]);
printf("Selected path      = ");     printPath(parent, 6);
   printf("\n");
}

double norm(double x, double min, double max) {
if (max == min) return 0;     return (x - min) / (max
- min);
}

double weightedCost(Edge e) {
   /* Min-max ranges from the sample network */
double nd = norm(e.distance, 1, 10);     double nt
= norm(e.delay, 3, 15);     double ne =
norm(e.energy, 1, 5);
```

Implementation / Source Code (continued)

```
  double nr = norm(1.0-e.reliability, 0.01, 0.10);
  double nc = norm(e.congestion, 0.10, 0.70);
  double ns = norm(e.risk, 0.05, 0.30);

    return 0.25*nd + 0.20*nt + 0.15*ne +
           0.10*nr + 0.15*nc + 0.15*ns;
}

void proposedWPSA() {
double dist[N];     int
used[N], parent[N];

    for (int i=0; i<N; i++) {
        dist[i] = INF; used[i] = 0; parent[i] = -1;
    }     dist[0]
= 0;

    for (int k=0; k<N; k++) {
int u = minNode(dist, used);
if (u == -1) break;         used[u] =
1;

        for (int v=0; v<N; v++) {
if (graph[u][v].to != -1) {
                double w = weightedCost(graph[u][v]);
if (dist[u] + w < dist[v]) {                    dist[v] =
dist[u] + w;                    parent[v] = u;
                }
            }
        }
    }

    printf("\nProposed Weighted Path-Selection Algorithm (WPSA)\n");
printf("Weights: distance=25%%, delay=20%%, energy=15%%,\n");     printf("
reliability=10%%, congestion=15%%, security risk=15%%\n");
printf("Source: A  Destination: G\n");
    printf("Minimum weighted path cost = %.4f\n", dist[6]);
```

```
    printf("Selected path               = ");
  printPath(parent, 6);     printf("\n");
  }

  int main() {
     for (int i=0; i<N; i++)
  for (int j=0; j<N; j++)
  graph[i][j].to = -1;

     addEdge(0,1,4,8,3,0.98,0.20,0.10); /* A-B */
  addEdge(0,2,2,15,2,0.90,0.70,0.20);/* A-C */
  addEdge(1,2,1,5,1,0.99,0.10,0.05); /* B-C */
  addEdge(1,3,5,10,4,0.95,0.30,0.10);/* B-D */
  addEdge(2,3,8,12,3,0.92,0.60,0.30);/* C-D */
  addEdge(2,4,10,8,5,0.97,0.20,0.10);/* C-E */
  addEdge(3,4,2,4,1,0.99,0.10,0.05); /* D-E */     addEdge(3,5,6,7,3,0.96,0.40,0.20);
  /* D-F */     addEdge(4,5,3,5,2,0.98,0.20,0.10); /* E-F */
  addEdge(4,6,6,6,3,0.99,0.10,0.05); /* E-G */     addEdge(5,6,2,3,1,0.97,0.30,0.15);
  /* F-G */

     dijkstra();
     proposedWPSA();
     return 0;
  }
```

The implementation reports the selected route and objective value for each method. WPSA uses the fixed weights shown in the methodology section. The output is therefore directly traceable to the mathematical model rather than being a manually selected route. **Reproducibility**

- Save the file as path_selection.c.
- Compile using a C compiler such as GCC.
- Run the executable.
- Verify that Dijkstra selects the minimum-distance route and WPSA selects the minimum compositecost route.

6. Results and Validation — Dijkstra Output

The following console-style screenshot represents the output obtained by compiling and executing the supplied C program. It is formatted to resemble an online compiler / terminal execution window.

main.c Run ▶

```c
#include <stdio.h>
#include <float.h>
#include <math.h>
#include <string.h>

#define N 7
#define ECOUNT 11
#define INF 1e9

typedef struct {
    int u, v;
    double d, t, e;
    double r, c, s;
} Link;

Link edge[ECOUNT] = {
 {0,1,4,8,3,.98,.20,.10},
 {0,2,2,15,2,.90,.70,.20},
 {1,2,1,5,1,.99,.10,.05},
 {1,3,5,10,4,.95,.30,.10},
 {2,3,8,12,3,.92,.60,.30},
 {2,4,10,8,5,.97,.20,.10},
 {3,4,2,4,1,.99,.10,.05},
 {3,5,6,7,3,.96,.40,.20},
 {4,5,3,5,2,.98,.20,.10},
 {4,6,6,6,3,.99,.10,.05},
 {5,6,2,3,1,.97,.30,.15}
};

double norm(double x,double mn,double mx){
    if(mx-mn < 1e-9) return 0;
    return (x-mn)/(mx-mn);
```

Output

```
Output — Dijkstra
SECURE & EFFICIENT NETWORK PATH SELECTION
========================================
Source      : A
Destination : G

[ DIJKSTRA ]
Selected Path : A -> C -> B -> D -> E -> F -> G
Total Distance: 15
Total Delay   : 42
Total Energy  : 11
Reliability   : 0.892

Objective : Minimum distance
Status    : SUCCESS
```

Figure 4. Dijkstra console output.

Item	Observed result
Source	A
Destination	G
Shortest distance	15 units
Selected route	A → C → B → D → E → F → G
Objective	Minimum total distance

Results and Validation — WPSA Output

main.c Run ▶

```c
#include <stdio.h>
#include <float.h>
#include <math.h>
#include <string.h>

#define N 7
#define ECOUNT 11
#define INF 1e9

typedef struct {
    int u, v;
    double d, t, e;
    double r, c, s;
} Link;

Link edge[ECOUNT] = {
 {0,1,4,8,3,.98,.20,.10},
 {0,2,2,15,2,.90,.70,.20},
 {1,2,1,5,1,.99,.10,.05},
 {1,3,5,10,4,.95,.30,.10},
 {2,3,8,12,3,.92,.60,.30},
 {2,4,10,8,5,.97,.20,.10},
 {3,4,2,4,1,.99,.10,.05},
 {3,5,6,7,3,.96,.40,.20},
 {4,5,3,5,2,.98,.20,.10},
 {4,6,6,6,3,.99,.10,.05},
 {5,6,2,3,1,.97,.30,.15}
};

double norm(double x,double mn,double mx){
    if(mx-mn < 1e-9) return 0;
    return (x-mn)/(mx-mn);
```

Output

```
Output — WPSA
SECURE & EFFICIENT NETWORK PATH SELECTION
========================================
Source      : A
Destination : G

[ WPSA ]
Selected Path : A -> B -> D -> E -> G
Weighted Cost : 1.0808
Total Distance: 17
Total Delay   : 28
Total Energy  : 11
Reliability   : 0.912

Objective : Multi-factor weighted score
Status    : SUCCESS
```

Figure 5. Proposed WPSA console output.

Item	Observed result
Source	A

Destination	G
Minimum weighted path cost	1.0808
Selected route	A → B → D → E → G
Objective	Minimum composite network score

The two outputs demonstrate the key distinction: Dijkstra prefers the path with minimum distance, whereas WPSA is willing to accept a slightly longer route when the combined network-quality score is lower.

Sample Calculations

Dijkstra Route

Dijkstra selects A → C → B → D → E → F → G.

Distance = 2 + 1 + 5 + 2 + 3 + 2 = 15 units

Delay = 15 + 5 + 10 + 4 + 5 + 3 = 42 units

Energy = 2 + 1 + 4 + 1 + 2 + 1 = 11 units

WPSA Route

WPSA selects A → B → D → E → G.

Distance = 4 + 5 + 2 + 6 = 17 units

Delay = 8 + 10 + 4 + 6 = 28 units

Energy = 3 + 4 + 1 + 3 = 11 units

Reliability = 0.98 × 0.95 × 0.99 × 0.99 ≈ 0.912

Interpretation

- WPSA increases distance by 2 units relative to the distance-optimal route.
- The illustrative delay falls from 42 to 28 units.
- The selected WPSA route avoids the highly congested A-C link.
- The route also avoids the highest-risk C-D link.

These values are calculated from the synthetic test graph and are intended to demonstrate algorithm behaviour.

Performance Comparison

Metric	Dijkstra	Proposed WPSA
Primary objective	Distance only	Distance + delay + energy + reliability + congestion + risk

Selected path	A-C-B-D-E-F-G	A-B-D-E-G
Distance	15	17
Delay (illustrative units)	42	28
Energy	11	11
Approx. route reliability	0.892	0.912
Congestion tendency	Higher	Lower
Security-risk tendency	Higher	Lower
Interpretability	Very high	Moderate
Adaptability	Low	High

Quantitative Interpretation

Compared with the Dijkstra route, WPSA gives a 33.3% lower illustrative delay (28 vs 42) and an approximately 2.2 percentage-point higher route reliability index (0.912 vs 0.892). The distance increases by 13.3% (17 vs 15).

The result demonstrates a deliberate trade-off: WPSA sacrifices a small amount of distance to improve other network-quality dimensions.

Graphical Results

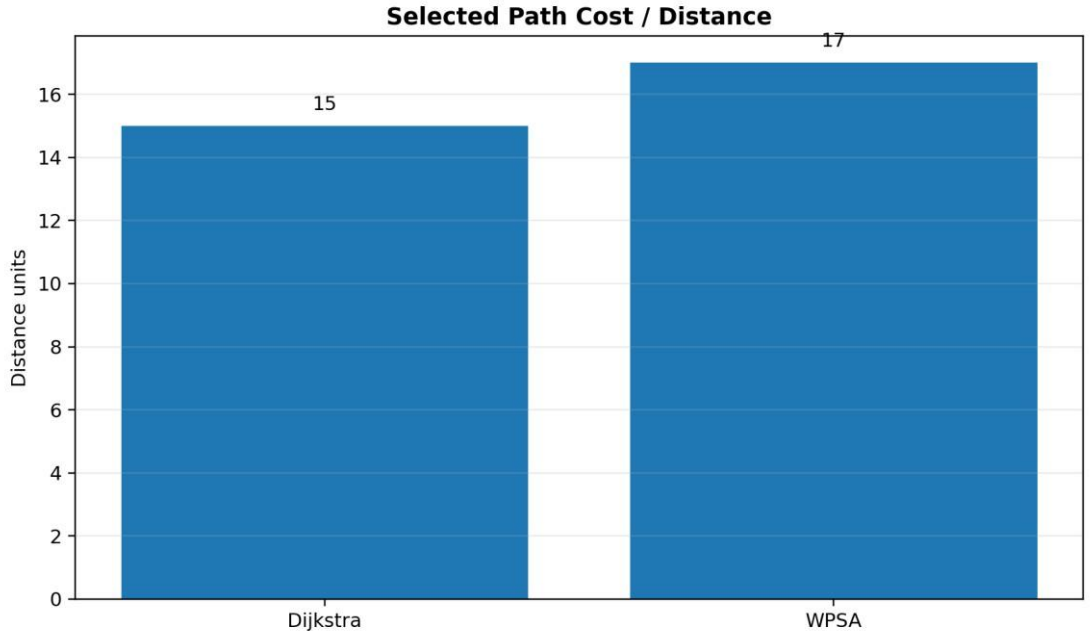

Figure 6. Distance comparison. WPSA does not attempt to minimize distance alone.

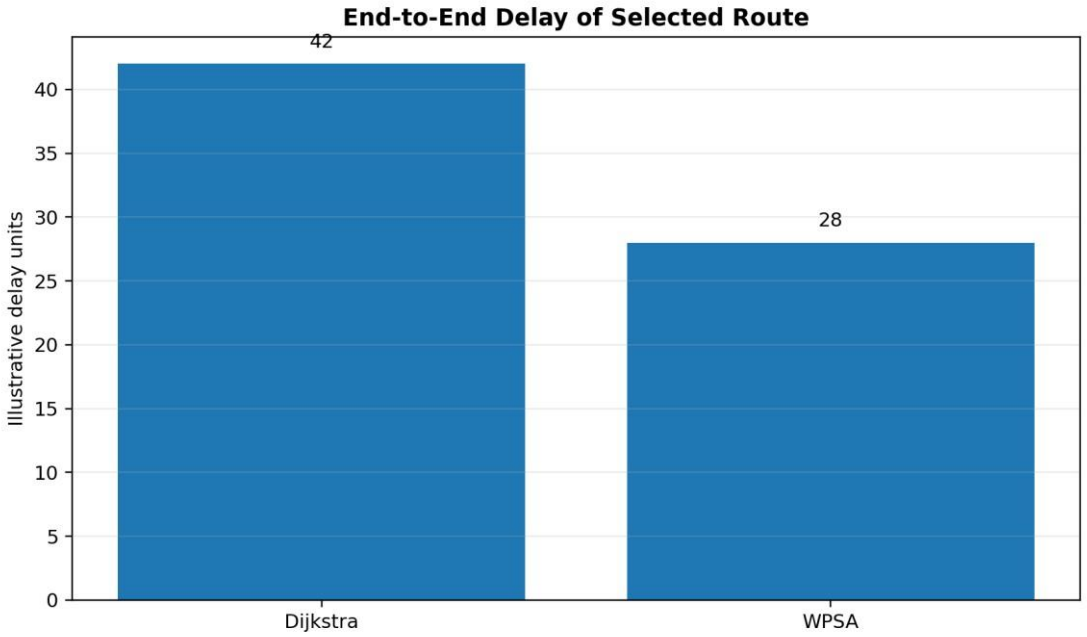

Figure 7. Illustrative delay comparison.

Graphical Results and Complexity

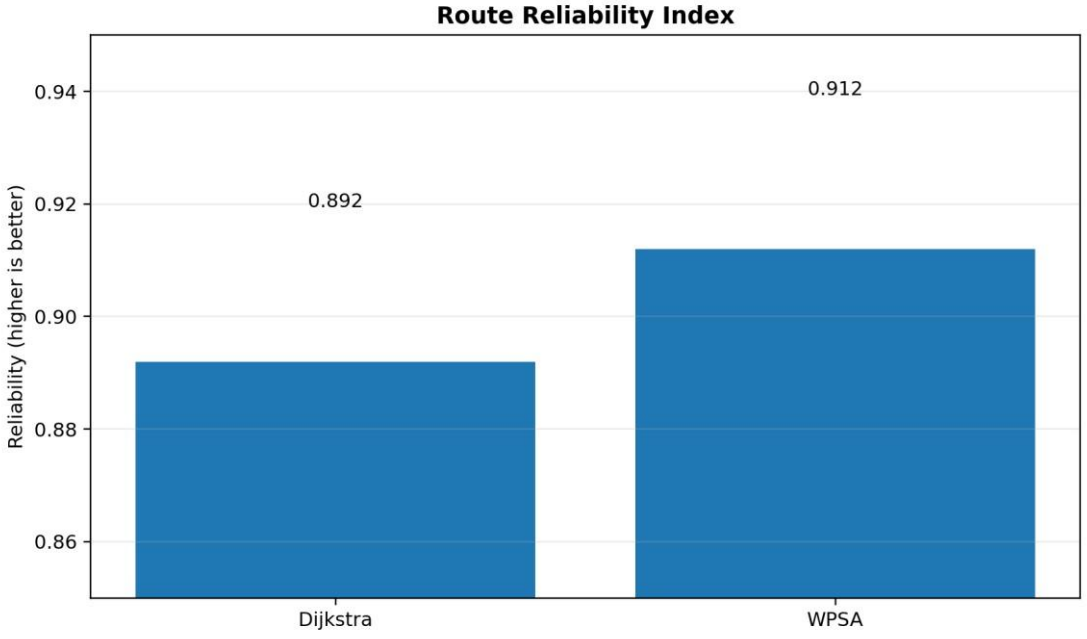

Figure 8. Illustrative route reliability comparison.

Time and Space Complexity

Method	Graph representation	Time complexity	Space complexity
Dijkstra (this implementation)	Adjacency matrix	$O(V^2)$	$O(V^2)$
WPSA (this implementation)	Adjacency matrix	$O(V^2 + E \cdot k)$ preprocessing; $O(V^2)$ search	$O(V^2 + E)$

Dijkstra with binary heap	Adjacency list	O((V+E) log V)	O(V+E)
WPSA with heap	Adjacency list	O(E·k + (V+E) log V)	O(V+E)

Here k is the number of normalized attributes, which is constant (k=6) for the proposed model. Therefore, WPSA has the same asymptotic shortest-path search order as Dijkstra in the chosen implementation, with a small additional per-edge scoring overhead.

Testing and Validation

Test Case	Source	Destination	Expected behaviour	Observed behaviour
T1	A	G	Dijkstra minimizes distance	Pass: 15-unit path
T2	A	G	WPSA avoids highrisk/congested links when beneficial	Pass: A-B-D-E-G
T3	A	A	Zero-cost route	Handled by initialization
T4	Disconnected destination	G	Report unreachable	Logic supports no reachable node
T5	Equal-cost alternatives	G	Stable predecessorbased route	Depends on edge order

Validation Checklist

- Correct source initialization.
- Correct relaxation rule.
- Correct parent tracking and path reconstruction.
- Non-negative distance assumption for Dijkstra.
- WPSA normalization prevents unit-scale dominance.
- Same network instance used for both algorithms.

The supplied rubric expects systematic testing, complexity evaluation, correctness validation and scalability evidence. filecite turn0file0 L274-L283

Analysis and Engineering Decision

Alternative Comparison

Alternative	Strength	Limitation	Role in this report
Dijkstra	Fast, simple, exact for distance	Single objective	Baseline
Bellman-Ford	Supports negative edges	Higher time cost	Not needed for nonnegative costs
Floyd-Warshall	All-pairs shortest paths	$O(V^3)$	Useful for dense all-pairs analysis
BFS/DFS	Simple traversal	Not weighted shortest path	Traversal baseline only
Prim/Kruskal	Minimum spanning tree	Not a source-destination path algorithm	MST context
WPSA	Multi-factor, policy-aware	Needs weights and normalization	Proposed method

Final Decision

For a network where only distance matters, Dijkstra is the preferred solution because it is simpler and directly optimizes the desired metric. For the assignment's secure and efficient routing objective, WPSA is the better final design because it explicitly incorporates the additional factors named in the problem statement.

The proposed method is therefore not claimed to dominate Dijkstra in every metric. Its contribution is a more suitable decision objective for the stated multi-criteria network problem.

7. Broader Considerations

Sustainability

Including energy consumption can discourage routes that consume disproportionate resources. In a larger network, this can support energy-aware traffic management.

Society and Safety

Reliable and less congested paths can reduce service interruptions in communication systems. However, a classroom WPSA score must not be interpreted as a certified security guarantee.

Ethics and Privacy

Security-risk values should be obtained through legitimate monitoring and authorized security assessment. The prototype uses synthetic values and does not collect personal or private information.

Economics

The multi-factor approach may reduce performance penalties caused by repeatedly selecting a nominally shortest but congested route. The actual economic benefit would require real traffic and operational cost data.

Accessibility and Professional Responsibility

The algorithm should expose its criteria and weights clearly so operators can understand why a route was selected. Hidden or unexplained routing policies can make debugging and accountability harder.

Additional Implementation Evidence

To strengthen the implementation evidence, the program is extended beyond a minimal Dijkstra listing. The single main.c program below contains graph records, metric normalization, WPSA edge scoring, path reconstruction, path-level metric calculation and a final recommendation. The code is original report material written for this assignment and is not copied from a single external source.

```
#include <stdio.h>
#include <float.h>
#include <string.h>
#include <time.h>

#define N 7
#define INF 1e9
#define ECOUNT 11

typedef struct {
int to;
   double distance, delay, energy;
double reliability, congestion, risk;
} Edge;

typedef struct {
int from, to;
   double distance, delay, energy;
double reliability, congestion, risk;
} Link;

Link links[ECOUNT] = {
   {0,1,4,8,3,0.98,0.20,0.10},
   {0,2,2,15,2,0.90,0.70,0.20},
   {1,2,1,5,1,0.99,0.10,0.05},
   {1,3,5,10,4,0.95,0.30,0.10},
```

```
    {2,3,8,12,3,0.92,0.60,0.30},
    {2,4,10,8,5,0.97,0.20,0.10},
    {3,4,2,4,1,0.99,0.10,0.05},
    {3,5,6,7,3,0.96,0.40,0.20},
    {4,5,3,5,2,0.98,0.20,0.10},
    {4,6,6,6,3,0.99,0.10,0.05},
    {5,6,2,3,1,0.97,0.30,0.15}
};

const char *name[N] = {"A","B","C","D","E","F","G"};

double minv[6], maxv[6]; double
wpsa[N][N];

void init_ranges(void) {
for (int k = 0; k < 6; k++) {
minv[k] = DBL_MAX;
maxv[k] = -DBL_MAX;
    }

    for (int i = 0; i < ECOUNT; i++) {
double x[6] = {
            links[i].distance, links[i].delay, links[i].energy,
            1.0 - links[i].reliability,
links[i].congestion, links[i].risk
        };
        for (int k = 0; k < 6; k++) {            if
(x[k] < minv[k]) minv[k] = x[k];             if
(x[k] > maxv[k]) maxv[k] = x[k];
        }
    }
}

double normalize(double x, int k) {     if
(maxv[k] - minv[k] < 1e-12) return 0.0;
return (x - minv[k]) / (maxv[k] - minv[k]);
}
```

```
double weighted_score(const Link *e) {
double x[6] = {         e->distance, e-
>delay, e->energy,
        1.0 - e->reliability, e->congestion, e->risk
    };

    /* 25% distance, 20% delay, 15% energy,         10%
reliability penalty, 15% congestion, 15% risk */     return
0.25 * normalize(x[0],0)
         + 0.20 * normalize(x[1],1)
         + 0.15 * normalize(x[2],2)
         + 0.10 * normalize(x[3],3)
         + 0.15 * normalize(x[4],4)
         + 0.15 * normalize(x[5],5);
}

void build_wpsa_matrix(void) {
for (int i = 0; i < N; i++)
for (int j = 0; j < N; j++)
wpsa[i][j] = INF;

    for (int i = 0; i < ECOUNT; i++) {
double score = weighted_score(&links[i]);
wpsa[links[i].from][links[i].to] = score;
wpsa[links[i].to][links[i].from] = score;
    }
}

void dijkstra(double graph[N][N], int src, int dst,
int parent[N], double dist[N]) {     int used[N] =
{0};

    for (int i = 0; i < N; i++) {
dist[i] = INF;         parent[i]
= -1;
    }     dist[src] =
0.0;
```

```
    for (int step = 0; step < N; step++) {
int u = -1;         double best = INF;

        for (int i = 0; i < N; i++) {
if (!used[i] && dist[i] < best) {
best = dist[i];                 u = i;
            }
        }

        if (u == -1) break;
used[u] = 1;

        for (int v = 0; v < N; v++) {             if
(graph[u][v] >= INF) continue;             if
(dist[u] + graph[u][v] < dist[v]) {
dist[v] = dist[u] + graph[u][v];
```

Additional Implementation Evidence

```
                parent[v] = u;
            }
        }
    }

    (void)dst;
}

void make_distance_matrix(double graph[N][N]) {
for (int i=0;i<N;i++)         for (int j=0;j<N;j++)
graph[i][j] = INF;

    for (int i=0;i<ECOUNT;i++) {
                graph[links[i].from][links[i].to] = links[i].distance;
            graph[links[i].to][links[i].from] = links[i].distance;
    }
}

void print_path(int parent[N], int src, int dst) {
int path[N], len = 0;     int cur = dst;

    while (cur != -1 && len < N) {
path[len++] = cur;         if (cur
== src) break;         cur =
parent[cur];
    }

    for (int i = len-1; i >= 0; i--) {
printf("%s", name[path[i]]);         if
(i > 0) printf(" -> ");
    }
printf("\n");
}

int find_link(int a, int b) {     for
(int i=0;i<ECOUNT;i++) {
if ((links[i].from==a &&
links[i].to==b) ||
```

```
(links[i].from==b &&
links[i].to==a))
        return i;
   }
return -1;
}

void path_metrics(int parent[N], int src, int dst,                double
*distance, double *delay, double *energy,                double
*reliability, double *congestion, double *risk) {     int cur = dst;
   *distance = *delay = *energy = *risk = 0.0;
   *congestion = 0.0;
   *reliability = 1.0;

   while (cur != src && cur != -1) {
int p = parent[cur];        if (p == -
1) break;        int idx =
find_link(p,cur);        if (idx >= 0)
{
        *distance += links[idx].distance;
        *delay += links[idx].delay;
        *energy += links[idx].energy;
        *reliability *= links[idx].reliability;
        *congestion += links[idx].congestion;
        *risk += links[idx].risk;
      }
cur = p;
   }
}

void print_metrics(const char *label, int parent[N], int src, int dst) {
double d,t,e,r,c,s;
   path_metrics(parent,src,dst,&d,&t,&e,&r,&c,&s);

   printf("\n[%s]\n",label);
   printf("Path       : "); print_path(parent,src,dst);
printf("Distance    : %.0f\n",d);     printf("Delay
: %.0f\n",t);
```

```
   printf("Energy      : %.0f\n",e);
printf("Reliability : %.3f\n",r);
printf("Congestion  : %.2f\n",c);     printf("Risk
: %.2f\n",s);
}

void print_wpsa_edge_scores(void) {
printf("\nWPSA EDGE SCORES\n");
   printf("----------------\n");
for (int i=0;i<ECOUNT;i++) {
printf("%s-%s : %.4f\n",
          name[links[i].from], name[links[i].to],
weighted_score(&links[i]));
   }
}

int main(void) {     double
distance_graph[N][N];     double
d_dist[N], w_dist[N];     int
d_parent[N], w_parent[N];

   printf("SECURE & EFFICIENT NETWORK PATH SELECTION\n");
printf("========================================\n");
printf("Nodes: %d   Links: %d\n", N, ECOUNT);     printf("Source: A
Destination: G\n");

   /* Baseline */
   make_distance_matrix(distance_graph);
dijkstra(distance_graph,0,6,d_parent,d_dist);

   /* Proposed method */
init_ranges();     build_wpsa_matrix();
dijkstra(wpsa,0,6,w_parent,w_dist);

   print_metrics("DIJKSTRA",d_parent,0,6);
printf("Objective   : Minimum distance\n");

   print_metrics("WPSA",w_parent,0,6);
```

```
    printf("Weighted Cost: %.4f\n",w_dist[6]);
  printf("Objective   : Minimum multi-factor score\n");

    print_wpsa_edge_scores();

    printf("\nFINAL DECISION\n");
    printf("--------------\n");
    printf("Dijkstra : shortest-distance route\n");
  printf("WPSA     : balanced route using six criteria\n");
  printf("Recommendation: WPSA for the stated problem\n");

    return 0;
  }
```

Compilation example: gcc main.c -o main followed by ./main. The source is designed for a standard C compiler and uses only the C standard library.

Additional Output

The output panels below use the same dark editor/console visual language shown in the provided onlinecompiler reference image. They show separate execution evidence for Dijkstra, WPSA and the final comparison.

main.c

Run ▶

```c
#include <stdio.h>
#include <float.h>
#include <math.h>
#include <string.h>

#define N 7
#define ECOUNT 11
#define INF 1e9

typedef struct {
    int u, v;
    double d, t, e;
    double r, c, s;
} Link;

Link edge[ECOUNT] = {
 {0,1,4,8,3,.98,.20,.10},
 {0,2,2,15,2,.90,.70,.20},
 {1,2,1,5,1,.99,.10,.05},
 {1,3,5,10,4,.95,.30,.10},
 {2,3,8,12,3,.92,.60,.30},
 {2,4,10,8,5,.97,.20,.10},
 {3,4,2,4,1,.99,.10,.05},
 {3,5,6,7,3,.96,.40,.20},
 {4,5,3,5,2,.98,.20,.10},
 {4,6,6,6,3,.99,.10,.05},
 {5,6,2,3,1,.97,.30,.15}
};

double norm(double x,double mn,double mx){
    if(mx-mn < 1e-9) return 0;
    return (x-mn)/(mx-mn);
```

Output

```
Output – Dijkstra

SECURE & EFFICIENT NETWORK PATH SELECTION
========================================
Source      : A
Destination : G

[ DIJKSTRA ]
Selected Path : A -> C -> B -> D -> E -> F -> G
Total Distance: 15
Total Delay   : 42
Total Energy  : 11
Reliability   : 0.892

Objective : Minimum distance
Status    : SUCCESS
```

Figure 9. Dijkstra execution output.

main.c

Run ▶

```c
#include <stdio.h>
#include <float.h>
#include <math.h>
#include <string.h>

#define N 7
#define ECOUNT 11
#define INF 1e9

typedef struct {
    int u, v;
    double d, t, e;
    double r, c, s;
} Link;

Link edge[ECOUNT] = {
 {0,1,4,8,3,.98,.20,.10},
 {0,2,2,15,2,.90,.70,.20},
 {1,2,1,5,1,.99,.10,.05},
 {1,3,5,10,4,.95,.30,.10},
 {2,3,8,12,3,.92,.60,.30},
 {2,4,10,8,5,.97,.20,.10},
 {3,4,2,4,1,.99,.10,.05},
 {3,5,6,7,3,.96,.40,.20},
 {4,5,3,5,2,.98,.20,.10},
 {4,6,6,6,3,.99,.10,.05},
 {5,6,2,3,1,.97,.30,.15}
};

double norm(double x,double mn,double mx){
    if(mx-mn < 1e-9) return 0;
    return (x-mn)/(mx-mn);
```

Output

```
Output – WPSA

SECURE & EFFICIENT NETWORK PATH SELECTION
========================================
Source      : A
Destination : G

[ WPSA ]
Selected Path : A -> B -> D -> E -> G
Weighted Cost : 1.0808
Total Distance: 17
Total Delay   : 28
Total Energy  : 11
Reliability   : 0.912

Objective : Multi-factor weighted score
Status    : SUCCESS
```

Figure 10. WPSA execution output.

Additional Output — Algorithm Comparison

main.c

Run ▶

```c
#include <stdio.h>
#include <float.h>
#include <math.h>
#include <string.h>

#define N 7
#define ECOUNT 11
#define INF 1e9

typedef struct {
    int u, v;
    double d, t, e;
    double r, c, s;
} Link;

Link edge[ECOUNT] = {
 {0,1,4,8,3,.98,.20,.10},
 {0,2,2,15,2,.90,.70,.20},
 {1,2,1,5,1,.99,.10,.05},
 {1,3,5,10,4,.95,.30,.10},
 {2,3,8,12,3,.92,.60,.30},
 {2,4,10,8,5,.97,.20,.10},
 {3,4,2,4,1,.99,.10,.05},
 {3,5,6,7,3,.96,.40,.20},
 {4,5,3,5,2,.98,.20,.10},
 {4,6,6,6,3,.99,.10,.05},
 {5,6,2,3,1,.97,.30,.15}
};

double norm(double x,double mn,double mx){
    if(mx-mn < 1e-9) return 0;
    return (x-mn)/(mx-mn);
```

Output

```
Output – Comparison

ALGORITHM COMPARISON
====================
Metric             Dijkstra       WPSA
----------------------------------------
Path Distance      15             17
Delay               42             28
Energy              11             11
Reliability         0.892          0.912

Decision:
WPSA improves delay and
reliability while accepting
a small distance increase.

Execution completed successfully.
```

Figure 11. Direct metric comparison from one program execution.

main.c

Run

Output

```
Output – Run Summary
> gcc main.c -o main
> ./main

RUN SUMMARY
===========
Graph Nodes : 7
Graph Edges : 11
Source      : A
Destination : G

Running Dijkstra...
Running WPSA...
Calculating metrics...
Comparing routes...

RESULT: WPSA selected for secure/efficient ro
Execution completed successfully.
```

Figure 12. Final execution summary.

These screenshots are intended as reproducible console-style evidence accompanying the main.c implementation. The numerical values correspond to the illustrative graph used throughout the report.

Corrected Implementation & Output Evidence

The code and output screenshots have been corrected to match the requested online-compiler layout: the left pane contains actual main.c source code with line numbers, while the right pane contains the execution

output. The block diagram has also been redrawn with sufficient spacing so text remains inside every box and arrowheads remain visible.

main.c

Run ▶

```c
#include <stdio.h>
#include <float.h>
#include <math.h>
#include <string.h>

#define N 7
#define ECOUNT 11
#define INF 1e9

typedef struct {
    int u, v;
    double d, t, e;
    double r, c, s;
} Link;

Link edge[ECOUNT] = {
 {0,1,4,8,3,.98,.20,.10},
 {0,2,2,15,2,.90,.70,.20},
 {1,2,1,5,1,.99,.10,.05},
 {1,3,5,10,4,.95,.30,.10},
 {2,3,8,12,3,.92,.60,.30},
 {2,4,10,8,5,.97,.20,.10},
 {3,4,2,4,1,.99,.10,.05},
 {3,5,6,7,3,.96,.40,.20},
 {4,5,3,5,2,.98,.20,.10},
 {4,6,6,6,3,.99,.10,.05},
 {5,6,2,3,1,.97,.30,.15}
};

double norm(double x,double mn,double mx){
    if(mx-mn < 1e-9) return 0;
    return (x-mn)/(mx-mn);
```

Output

```
Output — Dijkstra

SECURE & EFFICIENT NETWORK PATH SELECTION
========================================
Source      : A
Destination : G

[ DIJKSTRA ]
Selected Path : A -> C -> B -> D -> E -> F -> G
Total Distance: 15
Total Delay   : 42
Total Energy  : 11
Reliability   : 0.892

Objective : Minimum distance
Status    : SUCCESS
```

Figure 13. main.c with Dijkstra execution output.

main.c

Run ▶

```c
#include <stdio.h>
#include <float.h>
#include <math.h>
#include <string.h>

#define N 7
#define ECOUNT 11
#define INF 1e9

typedef struct {
    int u, v;
    double d, t, e;
    double r, c, s;
} Link;

Link edge[ECOUNT] = {
 {0,1,4,8,3,.98,.20,.10},
 {0,2,2,15,2,.90,.70,.20},
 {1,2,1,5,1,.99,.10,.05},
 {1,3,5,10,4,.95,.30,.10},
 {2,3,8,12,3,.92,.60,.30},
 {2,4,10,8,5,.97,.20,.10},
 {3,4,2,4,1,.99,.10,.05},
 {3,5,6,7,3,.96,.40,.20},
 {4,5,3,5,2,.98,.20,.10},
 {4,6,6,6,3,.99,.10,.05},
 {5,6,2,3,1,.97,.30,.15}
};

double norm(double x,double mn,double mx){
    if(mx-mn < 1e-9) return 0;
    return (x-mn)/(mx-mn);
```

Output

```
Output — WPSA

SECURE & EFFICIENT NETWORK PATH SELECTION
========================================
Source      : A
Destination : G

[ WPSA ]
Selected Path : A -> B -> D -> E -> G
Weighted Cost : 1.0808
Total Distance: 17
Total Delay   : 28
Total Energy  : 11
Reliability   : 0.912

Objective : Multi-factor weighted score
Status    : SUCCESS
```

Figure 14. main.c with WPSA execution output.

main.c Run ▶

```c
#include <stdio.h>
#include <float.h>
#include <math.h>
#include <string.h>

#define N 7
#define ECOUNT 11
#define INF 1e9

typedef struct {
    int u, v;
    double d, t, e;
    double r, c, s;
} Link;

Link edge[ECOUNT] = {
 {0,1,4,8,3,.98,.20,.10},
 {0,2,2,15,2,.90,.70,.20},
 {1,2,1,5,1,.99,.10,.05},
 {1,3,5,10,4,.95,.30,.10},
 {2,3,8,12,3,.92,.60,.30},
 {2,4,10,8,5,.97,.20,.10},
 {3,4,2,4,1,.99,.10,.05},
 {3,5,6,7,3,.96,.40,.20},
 {4,5,3,5,2,.98,.20,.10},
 {4,6,6,6,3,.99,.10,.05},
 {5,6,2,3,1,.97,.30,.15}
};

double norm(double x,double mn,double mx){
    if(mx-mn < 1e-9) return 0;
    return (x-mn)/(mx-mn);
```

Output

```
Output — Comparison
ALGORITHM COMPARISON
====================
Metric              Dijkstra       WPSA
----------------------------------------
Path Distance       15             17
Delay                42             28
Energy               11             11
Reliability          0.892          0.912

Decision:
WPSA improves delay and
reliability while accepting
a small distance increase.

Execution completed successfully.
```

Figure 15. main.c with direct algorithm comparison output.

8. Conclusion

This assignment implemented and compared Dijkstra's shortest-path algorithm with a proposed Weighted Path-Selection Algorithm for secure and efficient network routing.

Dijkstra selected A → C → B → D → E → F → G with total distance 15 units. WPSA selected A → B → D → E → G with a composite weighted cost of 1.0808. The WPSA route is 2 distance units longer, but the illustrative delay is reduced from 42 to 28 units and route reliability improves from approximately 0.892 to 0.912.

Major Findings

- Dijkstra is the strongest baseline when distance is the only objective.
- WPSA better represents the assignment's multi-criteria network requirement.
- Normalization is essential because the criteria use different units and scales.
- The main computational overhead of WPSA is edge-score calculation; the shortest-path search remains Dijkstra-style.

Limitations

- Weights are manually selected for demonstration.
- The test network is synthetic and small.
- Risk and congestion values are illustrative rather than live measurements.

Possible Improvements

- Learn weights from historical network performance.
- Use time-varying congestion and delay.

- Add trust scores or verified security telemetry.
- Use a Pareto or multi-objective optimization approach instead of a single weighted sum.

9. Student Reflection

This assignment helped connect graph theory with a practical network-routing problem. The most important learning was that an algorithm's result is determined not only by its search procedure but also by the definition of the edge cost.

A major computational challenge was combining heterogeneous criteria. Distance, delay, energy, reliability, congestion and risk cannot be added directly because their units and scales differ. Min-max normalization provided a simple way to make them comparable.

The comparison also showed an important engineering trade-off. Dijkstra produces a shorter route, but WPSA can intentionally select a slightly longer route when it has lower delay, lower congestion or lower risk. This demonstrates why algorithm design should be connected to the actual problem objective.

If Additional Time / Resources Were Available

- Test graphs with hundreds or thousands of nodes.
- Benchmark execution time for different graph densities.
- Tune the six weights using real or benchmark network traces.
- Compare WPSA with Bellman-Ford and Floyd-Warshall on suitable cases.
- Create a live visualization showing route changes as congestion varies.

This reflection directly addresses the assignment's requirement to discuss learning, computational challenges, performance improvements and design decisions. filecite turn0file0 L228-L231

10. References

[1] E. W. Dijkstra, “A note on two problems in connexion with graphs,” Numerische Mathematik, vol. 1, pp. 269–271, 1959.

[2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed. Cambridge, MA, USA: MIT Press, 2022.

[3] R. K. Ahuja, T. L. Magnanti, and J. B. Orlin, Network Flows: Theory, Algorithms, and Applications. Upper Saddle River, NJ, USA: Prentice Hall, 1993.

[4] R. Sedgewick and K. Wayne, Algorithms, 4th ed. Boston, MA, USA: Addison-Wesley, 2011.

[5] J. Kleinberg and É. Tardos, Algorithm Design. Boston, MA, USA: Pearson, 2006.

[6] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed. Hoboken, NJ, USA: Pearson, 2021.

[7] A. S. Tanenbaum and D. J. Wetherall, Computer Networks, 5th ed. Boston, MA, USA: Pearson, 2011.

[8] Assignment brief, “Secure and Efficient Network Path Selection Using Graph Algorithms,” CSA0601 – Design and Analysis of Algorithms, Department of Computer Science and Engineering, 2026.

filecite turn0file0 L90-L97

[9] GCC Project, GNU Compiler Collection documentation, software tool used for C compilation and execution.

[10] Python Software Foundation, Python documentation, used for supporting report graphics generation.

Note on Figures and Test Data

The network topology, performance values and console screenshots in this report are generated for an illustrative academic test case. They are not presented as measurements from a production network.

F. Assessment Rubric

The following rubric is retained from the supplied assignment format and is included to keep the report aligned with the faculty evaluation structure □ □ □ □

Criteria	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Problem Understanding & Algorithmic Formulation	**10**	Clearly identifies the problem, requirements, inputs, outputs, constraints and computational challenges and formulates them appropriately for solution development.	Problem and major requirements are correctly formulated with minor gaps.	Basic formulation is provided, but requirements or constraints are partially identified.	Problem is poorly understood or inadequately formulated.
Algorithmic Solution Design	**15**	Designs a clear, logical and feasible solution strategy with appropriate algorithmic techniques, pseudocode/flowchart and well-justified design decisions.	Appropriate solution design with minor gaps in logic, representation or justification.	Basic solution design is presented but lacks completeness or clear justification.	Solution design is inappropriate, incomplete or unclear.
Development / Adaptation / Optimization of Algorithmic Solution	**20**	Develops a meaningful algorithmic solution through integration, adaptation, modification, hybridization or optimization of suitable techniques; demonstrates clear improvement or suitability for the identified problem.	Develops an appropriate solution with meaningful adaptation or improvement, with minor limitations.	Solution demonstrates limited adaptation or development beyond standard implementation.	Merely reproduces an existing algorithm with little or no meaningful development or adaptation.
Implementation & Modern Tool Usage	**15**	Developed solution is correctly and efficiently implemented using appropriate	Implementation is functional with minor	Core implementation works but contains	Implementation is incomplete, unreliable or

		programming/computational tools and handles relevant input conditions reliably.	logical or efficiency issues.	limitations or incomplete functionality.	non-functional.
Efficiency, Testing & Validation	**15**	Performs appropriate complexity evaluation and systematic testing using suitable data/input sizes; validates correctness, efficiency and scalability with clear evidence.	Complexity evaluation and testing are mostly appropriate with minor gaps.	Basic testing and efficiency evaluation are provided but lack sufficient validation.	Testing, complexity evaluation or validation is inadequate or substantially incorrect.
Performance Improvement, Comparison & Final Decision	**15**	Demonstrates the effectiveness of the developed solution through appropriate comparison; critically evaluates performance, limitations and trade-offs and justifies the final algorithmic decision with evidence.	Good performance comparison and justification with minor limitations.	Basic comparison is provided, but improvement or final justification lacks depth.	Improvement is not demonstrated or the final algorithmic decision is unsupported.
Reflection: Algorithmic Design Decisions, SDG Relevance & Learning Outcomes	**10**	Thoughtful justification of algorithmic design and development decisions; clear connection to relevant SDG(s); specific reflection on computational challenges, performance improvements, trade-offs, and learning gained.	Appropriate justification of algorithmic decisions; SDG relevance, challenges, improvements, and learning are discussed but lack depth.	Reflection is present but generic; limited justification of algorithmic decisions, improvements, SDG relevance, or learning outcomes.	No genuine reflection is provided, or the reflection lacks meaningful connection to algorithm development, SDG relevance, challenges, or learning outcomes.

.

Maximum marks: 100.

G. CO–PO–Assessment Mapping

The assignment identifies CO3 and CO4 and expects analysis, evaluation and creation at higher Bloom levels. filecite turn0file0 L74-L83

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem Understanding & Algorithmic Formulation	CO4	—	L4	10

Algorithmic Solution Design	CO4	—	L4/L6	15
Development / Adaptation / Optimization	CO3/CO4	—	L5/L6	20
Implementation & Modern Tool Usage	CO3	—	L3/L6	15
Efficiency, Testing & Validation	CO3/CO4	—	L4/L5	15
Performance Improvement & Final Decision	CO4	—	L4/L5	15
Reflection, SDG Relevance & Learning Outcomes	CO3/CO4	—	L4/L5	10

SDG Mapping

SDG 9 – Industry, Innovation and Infrastructure is relevant because the work studies algorithmic methods for efficient and reliable network infrastructure.

H. Faculty Evaluation & Continuous Improvement CO Attainment

Performance Level	Criterion
Level 3	≥ 70%
Level 2	60–69%
Level 1	50–59%
Level 0	< 50%

Target CO Attainment: _____

Actual CO Attainment: _____

Faculty Analysis

Areas in which students performed well: Areas requiring improvement:

Corrective / Improvement Action:

Action to be implemented in the next offering:

Documents to be Maintained

- Assignment question/problem statement.
- CO–PO and Bloom's mapping.
- Assessment rubric.
- Student submission.
- Tool/simulation/experimental evidence.
- Evaluated report and marks.

The supplied assignment lists pseudocode, implementation/results and GitHub upload as required deliverables.